

Reviewer Pack — Minimal Order of Elliptic Curves and Resolution Proof Width ...

Entry ec62039c046b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 19:37:19 UTC

Minimal Order of Elliptic Curves and Resolution Proof Width Correlation

Entry ID: ec62039c046b **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 19:37:19 UTC

1. Conjecture

Field A (mathematical branch): Elliptic Curve Theory **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF φ , the minimal order of an elliptic curve over a finite field that has a point in common with every line in the projective plane representation of the polynomials forming φ is linearly correlated with its resolution proof width $w(\varphi)$, such that $\text{minimal_order}(\varphi) = \Theta(w(\varphi))$.

Rationale (proposer's reasoning):

Elliptic curves provide a geometric interpretation for certain algebraic structures, and their properties can reveal deep number-theoretic connections. This conjecture bridges the gap between the abstract nature of elliptic curves and the concrete computational complexity of resolution proofs, potentially exposing new structural insights into both fields.

Taxonomy category: ELLIPTIC_CURVE_THEORY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 5af65f381241e2d3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each CNF φ , if the correlation coefficient between $\text{minimal_order}(\varphi)$ and $w(\varphi)$ is ≥ 0.8 AND the mean of $\text{minimal_order}(\varphi)$ for all trials falls within 3 standard deviations of the expected value under the null hypothesis, then support the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal order elliptic curves AND resolution proof complexity - projective plane representation CNF AND elliptic curve over finite field - resolution proof width correlation with minimal order elliptic curves

Top relevant hits considered: - [http://arxiv.org/abs/2110.06718v2] The 2-parity conjecture for elliptic curves with isomorphic 2-torsion - [http://arxiv.org/abs/1807.03761v3] The second moment of the number of integral points on elliptic curves is bounded - [http://arxiv.org/abs/1203.0809v2] Average rank of elliptic curves - [http://arxiv.org/abs/1802.06333v3] Explicit equations of a fake projective plane - [http://arxiv.org/abs/1101.2456v3] Polynomial representations and categorifications of Fock Space - [http://arxiv.org/abs/0711.0795v4] Finite-Dimensional Representations of Hyper Loop Algebras Over Non-Algebraically Closed Fields - [http://arxiv.org/abs/0804.1861v2] Elliptic curve configurations on Fano surfaces - [http://arxiv.org/abs/1704.02056v2] Counting elliptic curves with bad reduction over a prescribed set of primes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
```

```

        C[i][j] += A[i][k] * B[k][j]

    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = Fraction(0, 1)
    sign = 1
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        det += sign * A[0][i] * determinant(submatrix)
        sign *= -1
    return det

def inverse(A):
    n = len(A)
    I = [[Fraction(1, 0) if i == j else Fraction(0, 1) for j in range(n)] for i in range(n)]
    A_augmented = [row + col for row, col in zip(A, I)]
    A_echelon = gaussian_elimination(A_augmented)
    n = len(A_echelon)
    if any(abs(A_echelon[i][i]) < 1e-9 for i in range(n)):
        raise ValueError("Singular matrix")
    A_inv = [[A_echelon[i][n+j] for j in range(n)] for i in range(n)]
    return A_inv

def projective_plane_representation(cnf):
    n = len(cnf)
    lines = []
    for clause in cnf:
        line = set()
        for literal in clause:
            if literal > 0:
                line.add((literal, 1))
            else:
                line.add((-literal, -1))
        lines.append(line)
    return lines

def minimal_order(cnf):
    n = len(cnf)
    lines = projective_plane_representation(cnf)
    A = [[0] * (n+1) for _ in range(n+1)]
    for i in range(n):
        for j in range(i+1, n):
            if any((x, y) in lines[i] and (-x, -y) in lines[j] for x, y in [(1, 0), (0, 1), (-1, 0)]):
                A[i][j] = A[j][i] = 1
    A[n][n] = 1
    try:
        inv_A = inverse(A)
        minimal_order_value = sum(inv_A[i][i] for i in range(n))
        return minimal_order_value
    except ValueError:
        return float('inf')

def resolution_proof_width(cnf):

```

Simplified DPLL solver to estimate width

```
def dpll(clauses, assignment):
    if not clauses:
        return 1
    unit_clause = next((c for c in clauses if len(c) == 1), None)
    if unit_clause:
        literal = unit_clause[0]
        new_assignment = assignment.copy()
        new_assignment[literal] = True
        new_clauses = [c for c in clauses if literal not in c and -literal not in c]
        return dpll(new_clauses, new_assignment) + 1
    pure_literal = next((l for l in range(1, n+1) if (l not in assignment and -l not in assignment)), None)
    if pure_literal is not None:
        new_assignment = assignment.copy()
        new_assignment[pure_literal] = True
        new_clauses = [c for c in clauses if pure_literal not in c and -pure_literal not in c]
        return dpll(new_clauses, new_assignment) + 1
    literal = next((l for l in range(1, n+1) if l not in assignment), None)
    new_assignment_true = assignment.copy()
    new_assignment_true[literal] = True
    new_clauses_true = [c for c in clauses if literal not in c and -literal not in c]
    width_true = dpll(new_clauses_true, new_assignment_true) + 1
    new_assignment_false = assignment.copy()
    new_assignment_false[-literal] = True
    new_clauses_false = [c for c in clauses if -literal not in c and literal not in c]
    width_false = dpll(new_clauses_false, new_assignment_false) + 1
    return max(width_true, width_false)

cnf = [[-l if l < 0 else l for l in clause] for clause in cnf]
assignment = {}
return dpll(cnf, assignment)
```

```
def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    cnf = [[random.randint(1, n) for _ in range(random.randint(2, n))] for _ in range(n)]
    minimal_order_value = minimal_order(cnf)
    resolution_width = resolution_proof_width(cnf)
    return {
        "metric_name": "minimal_order",
        "metric_value": minimal_order_value,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": minimal_order_value <= resolution_width * 2 and minimal_order_value >= resolution_width,
        "counterexample": ""
    }
```

```
if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```
mean_value = sum(res["metric_value"] for res in results) / len(results)
std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
```

```

support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif any(not res["conjecture_holds"] for res in results):
    first_failing_seed = next(seed for seed, res in zip(seeds, results) if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18ffb5b4.py", line 144, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18ffb5b4.py", line 129, in run_trial
    minimal_order_value = minimal_order(cnf)
                          ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18ffb5b4.py", line 87, in minimal_order
    inv_A = inverse(A)
            ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_18ffb5b4.py", line 55, in inverse
    I = [[Fraction(1, 0) if i == j else Fraction(0, 1) for j in range(n)] for i in range(n)]
          ^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(1, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data to evaluate the conjecture. | next: Investigate and fix the crash in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13626
2	preregistration	ollama_remote	glm4:latest	0	0	11481
3	novelty	ollama_remote	glm4:latest	0	0	8339
4	novelty	ollama_remote	glm4:latest	0	0	9698
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10702
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20831
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20112
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18682
9	judge	ollama_remote	glm4:latest	0	0	15907

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129378 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/ec62039c046b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ec62039c046b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ec62039c046b.tar.gz` (if generated)